

Clase 7

# Node.js

---

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra  
aaossa@ing.puc.cl

1. Recuerden la Tarea 1
2. Resultados Control 1
3. Encuesta de Carga Académica

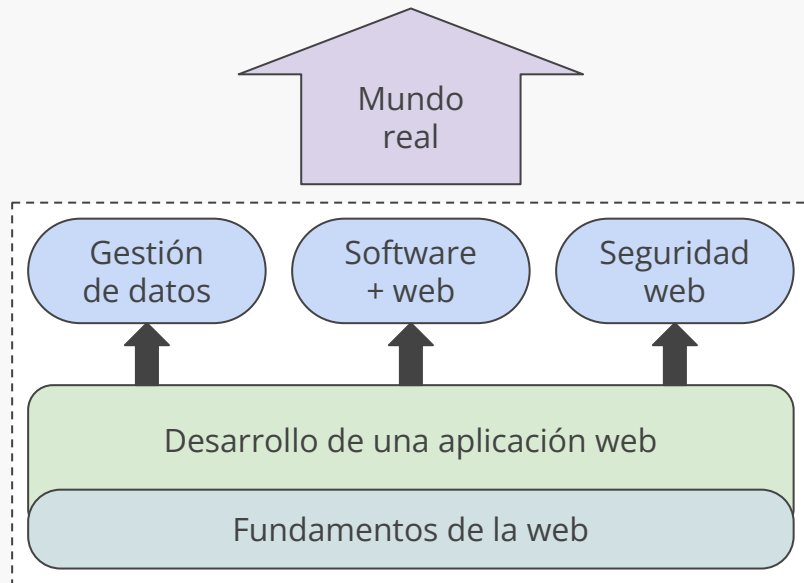
# Anuncios

---

# Contenidos del curso

---

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real



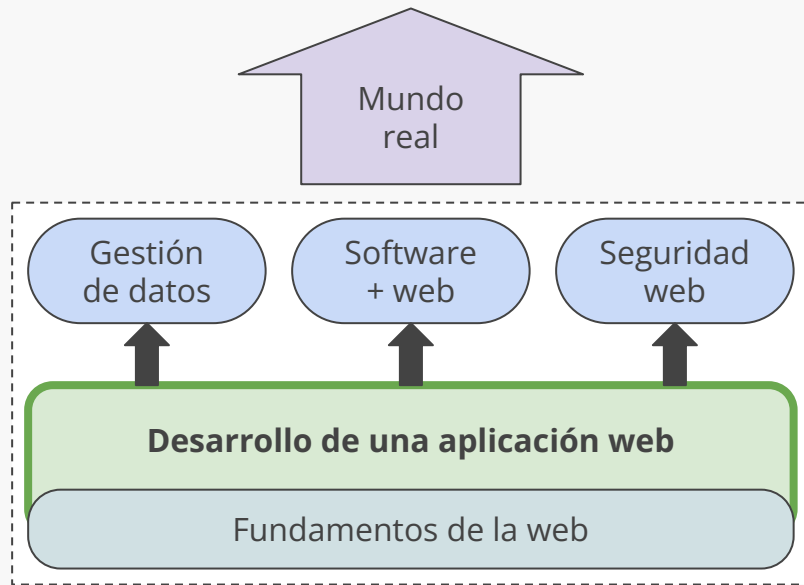
# Contenidos del curso

---

## Desarrollo de una aplicación web

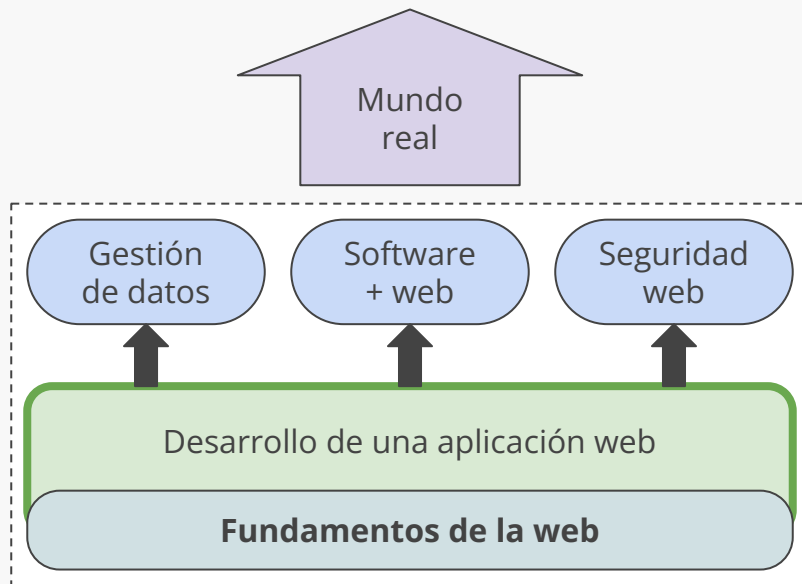
Veremos herramientas como React, **Node.js**, Koa y Express, y técnicas como CSS avanzado y el uso de *Design Systems*

... para desarrollar **aplicaciones modernas** tanto en el lado del cliente (front-end) como en el lado del servidor (back-end)



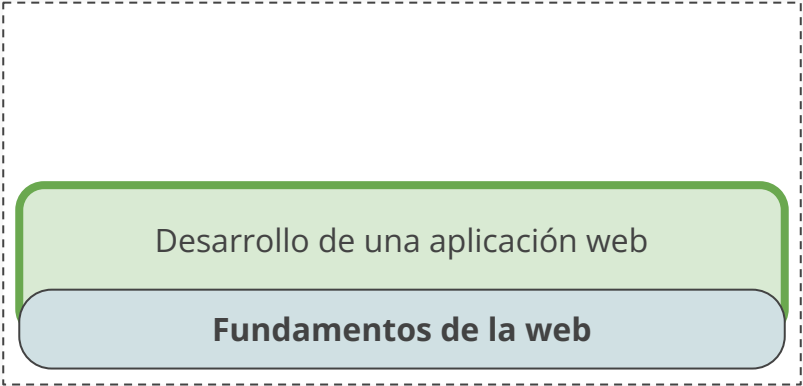
# Contenidos del curso

---



# Contenidos del curso

---

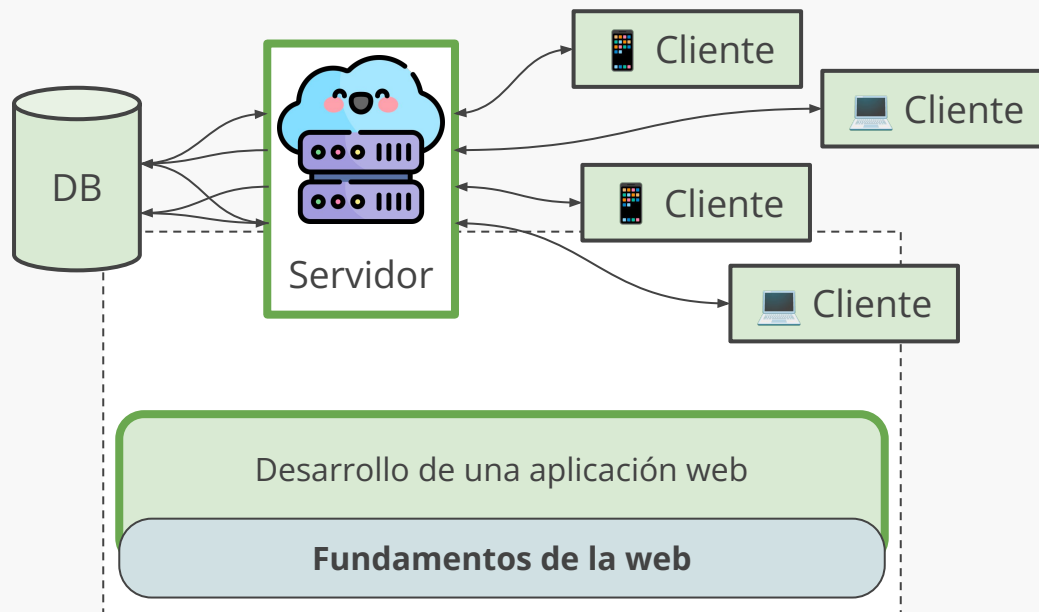


Desarrollo de una aplicación web

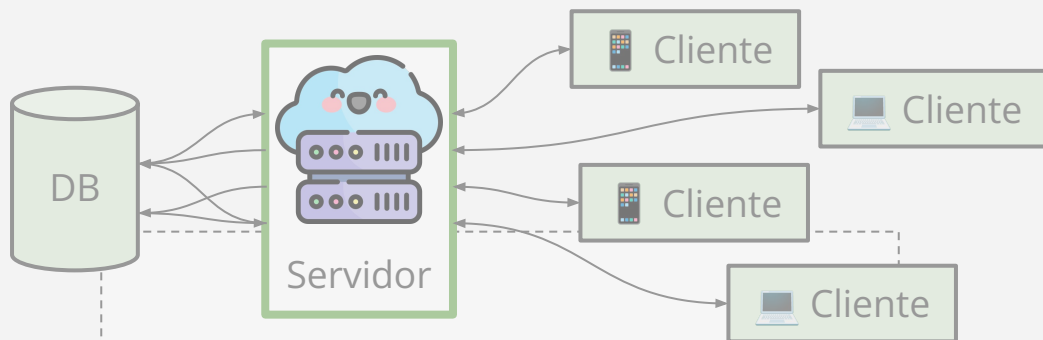
**Fundamentos de la web**

# Motivación

---



# Motivación

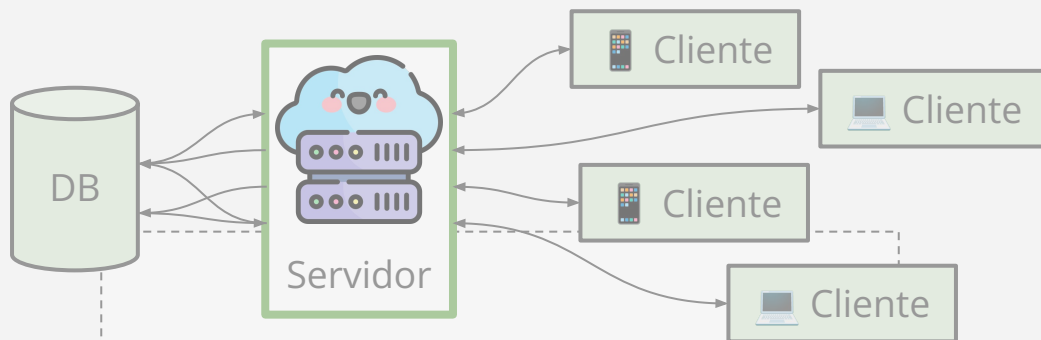


Necesitamos un lenguaje que responda a eventos, ejecute un *callback* a la vez y soporte *closures*

Fundamentos de la web



# Motivación

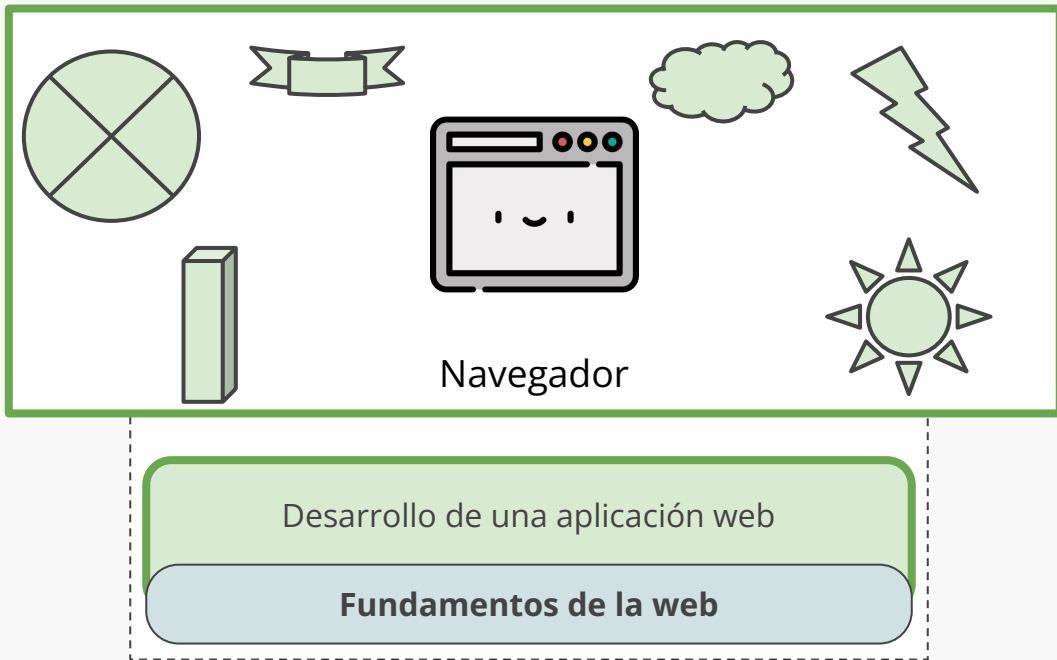


# JavaScript

Fundamentos de la web

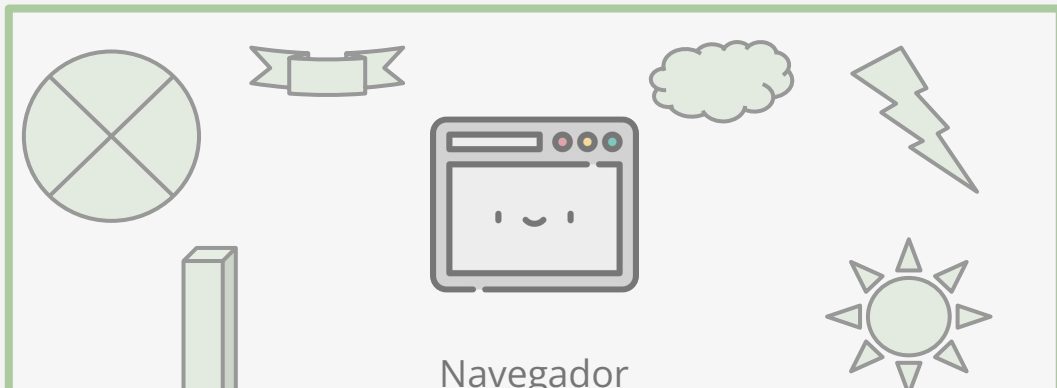
# Motivación

---



# Motivación

---



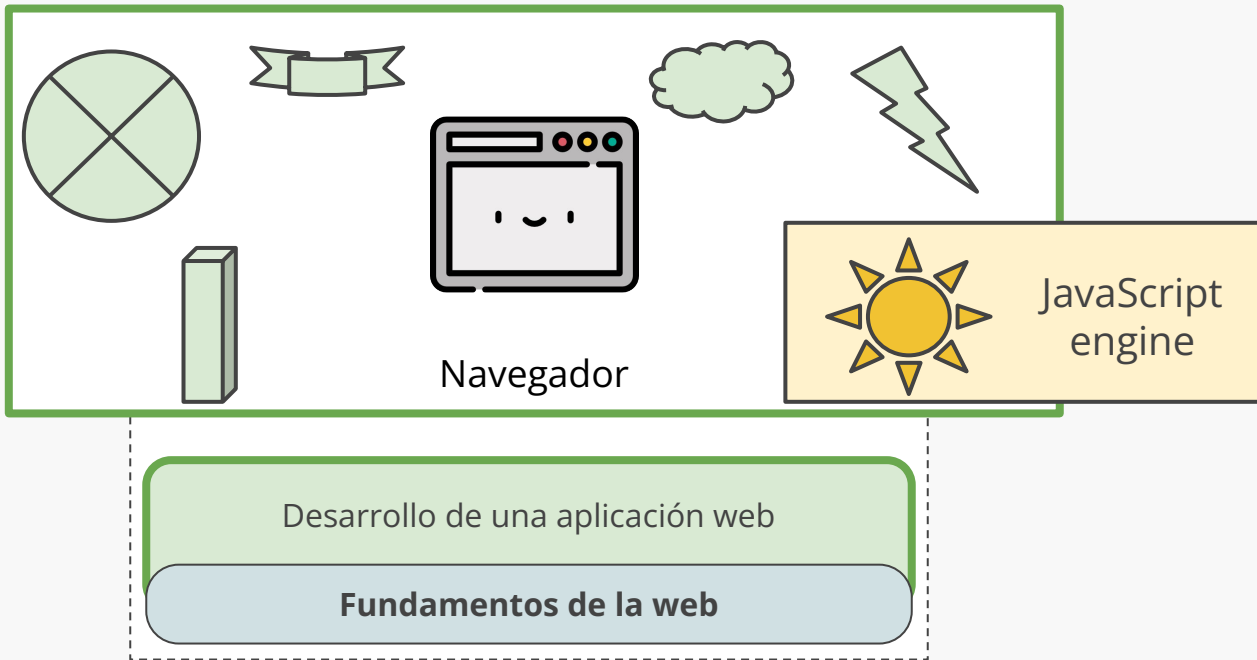
Navegador

**Pregunta#**¿Cómo podemos ejecutar JavaScript fuera del navegador?

Fundamentos de la web

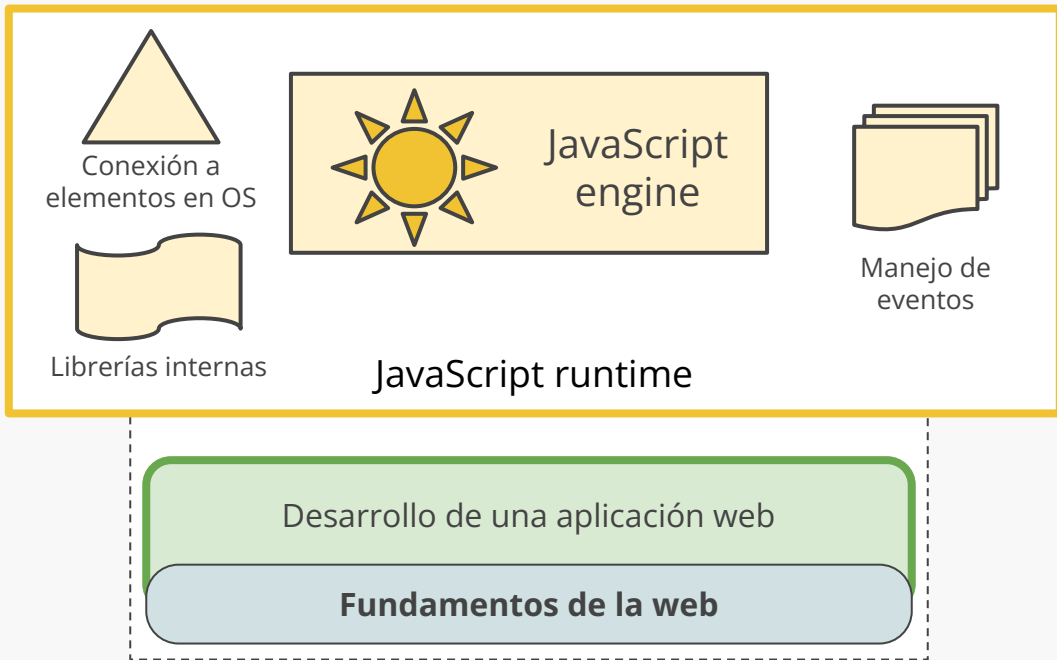
# Motivación

---



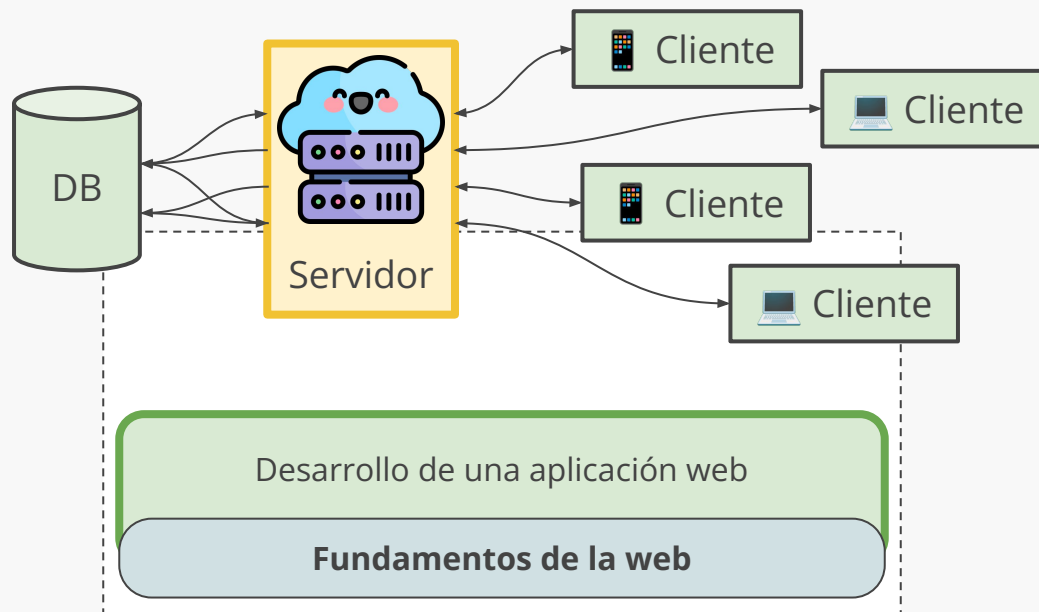
# Motivación

---



# Motivación

---



**Menti#***(No se me ocurrió un hook para hoy)*

# ¿Cuánto sabes de Node.js?

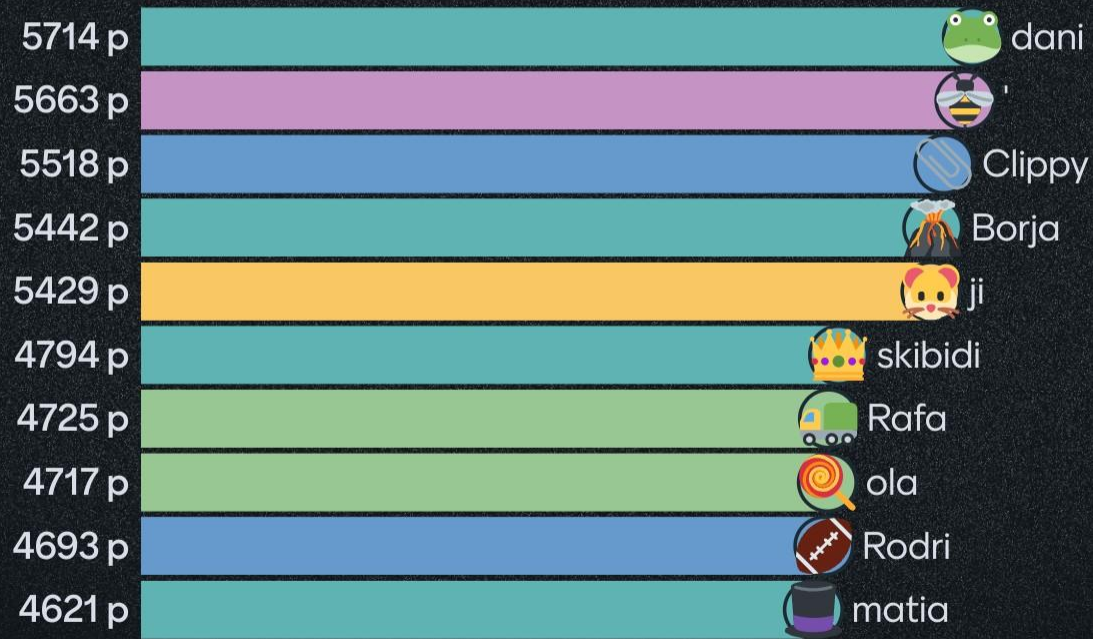




# ¿Te gusta hacer Kahoot?



# Quiz leaderboard

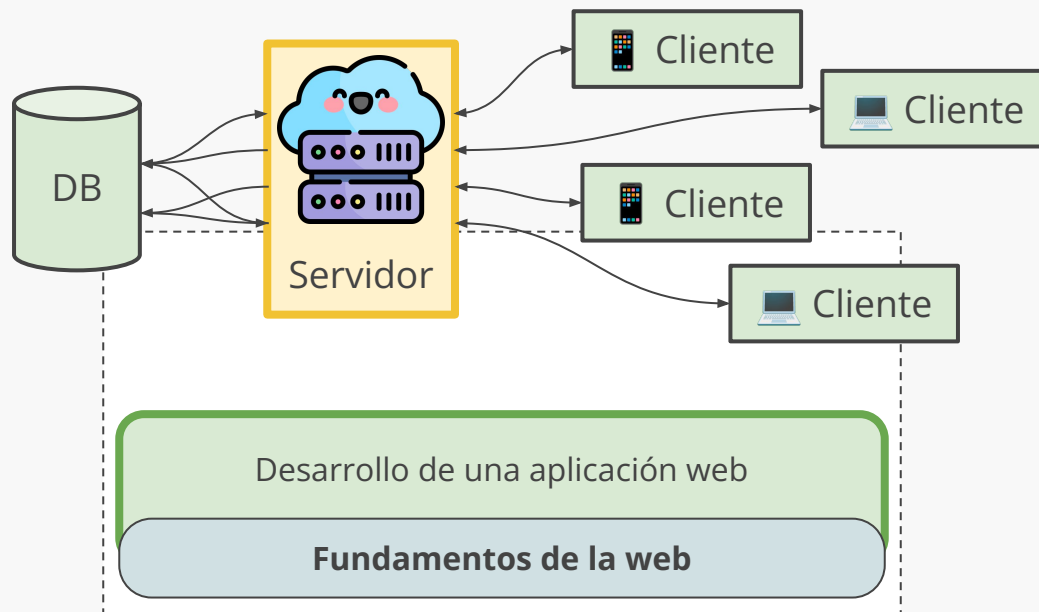






# Motivación

---



# ¿Dónde ya ejecutamos JavaScript?

---



Google Chrome



Microsoft Edge



Apple Safari



Mozilla Firefox



Opera



Brave

# ¿Cómo los navegadores ejecutan JavaScript?

---

V8 JavaScript Engine

[v8/v8 - Git at Google](#)



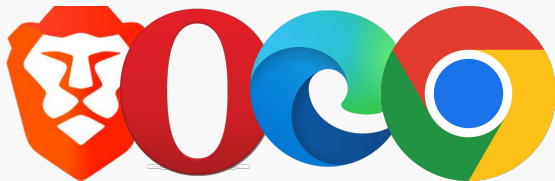
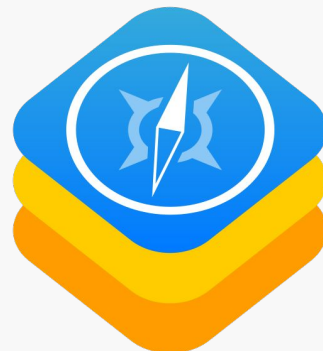
SpiderMonkey

[spidermonkey.dev](#)



JavaScriptCore

[WebKit/WebKit @ GitHub](#)



# ¿Cómo los navegadores ejecutan JavaScript?

V8 JavaScript Engine

[v8/v8 - Git at Google](#)

SpiderMonkey

[spidermonkey.dev](#)

JavaScriptCore

[WebKit/WebKit @ GitHub](#)

## ¿Qué tiene que ver con Node.js?



Más información: [Browser Engines... Chromium, V8, Blink? Gecko? WebKit? | by Jonathan Biro | Medium](#)

# Agenda

---

✨ Node.js



# Node.js

---

[Node.js](#) es un entorno de ejecución para JavaScript, que es *open source* y multiplataforma, **construido sobre V8** (el motor de Chrome)

Incluye los componentes necesarios para ejecutar código JavaScript. El **motor** V8 se encarga de **interpretar** el código JavaScript, mientras que Node combina distintas APIs para funcionar como *backend* (***runtime environment***)



# “Prehistoria” de Node.js

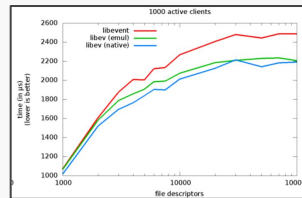
1995

JavaScript nace como lenguaje de programación para el *browser*



2007

Librería libev ofrece un *event loop* de alta *performance*, que incluye una *priority queue*



2008

Nace V8, el motor de JavaScript *open source* Chrome



2009

Ryan Dahl crea Node.js al combinar V8 con APIs I/O de bajo nivel (libUV\* y C++) para ejecutar JS en un ambiente como un servidor



# Milestones de NodeJS

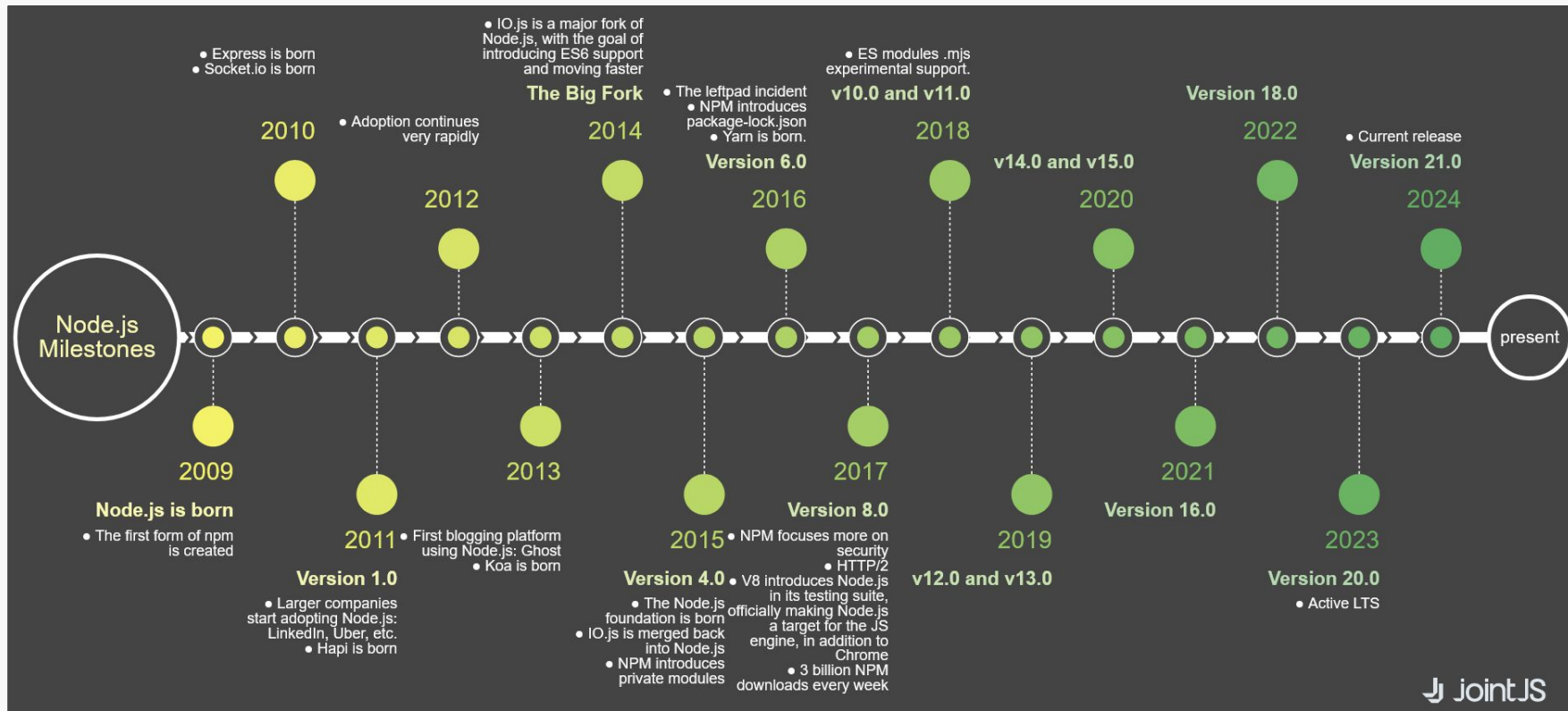


Figura: [NodeJS Milestones Timeline](#) | [JointJS](#) (editable en [Codepen](#))

# Versionamiento de NodeJS

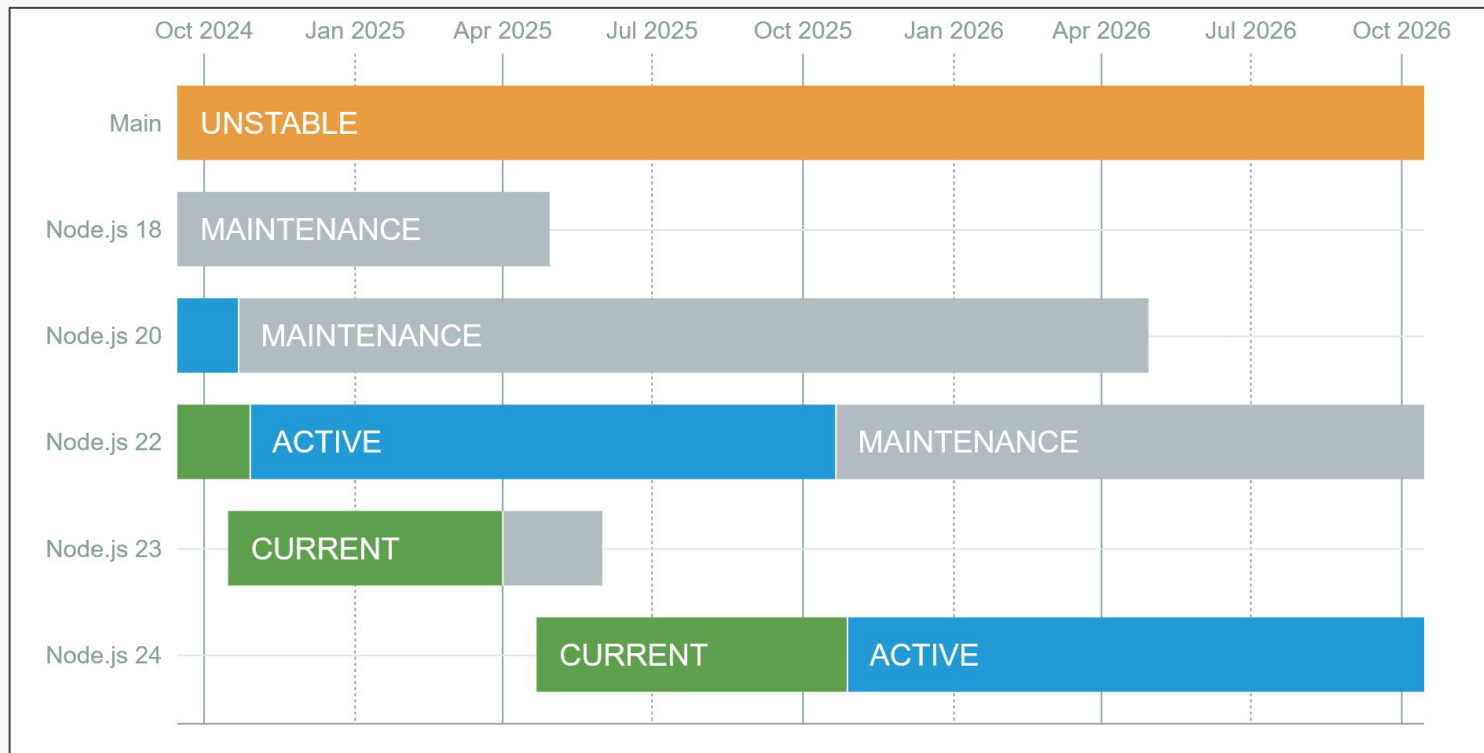
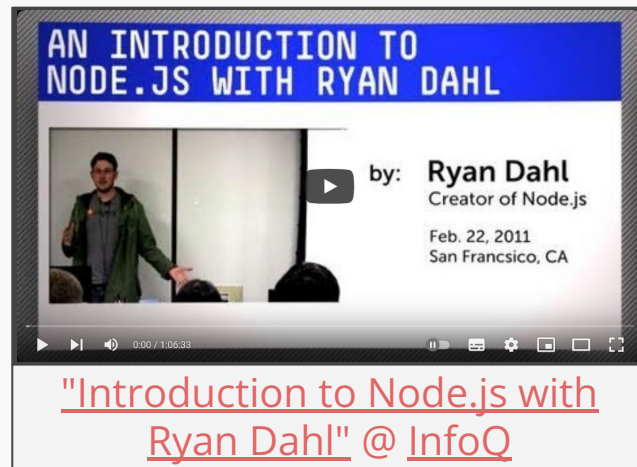


Figura: [Node.js Releases](#)

# Un extra...

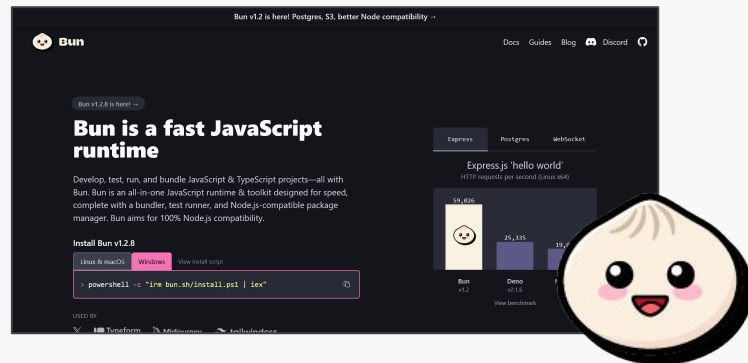
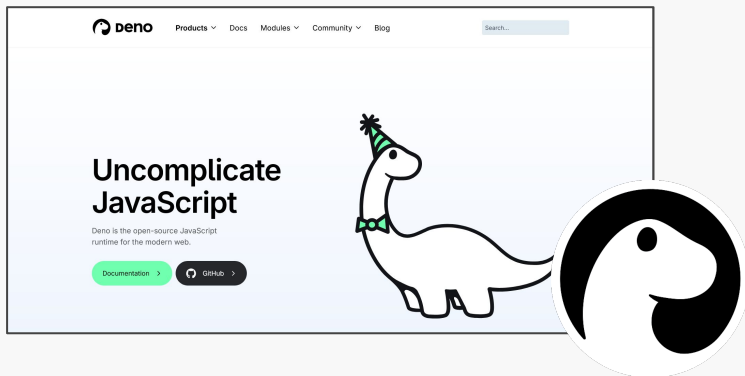
Inicialmente, Node.js fue presentado por Ryan Dahl en **JSConf**, una serie de conferencias para la comunidad de JavaScript. La introducción de Node.js al mundo fue en 2009, pero también hay una presentación sobre una versión temprana (v0.4.0). Ambas charlas nos permiten conocer el propósito y funcionamiento de Node.js de parte de quien lo creó:



# ¿Hay más?

Node.js no es el único *runtime* de JavaScript:

- **Deno** es un runtime moderno para JavaScript y TypeScript creado por Ryan Dahl, el creador original de Node.js. Fue diseñado para superar algunas de las limitaciones y problemas de seguridad de Node.js.
- **Bun** es un runtime de JavaScript y TypeScript diseñado para ser rápido. Está basado en el motor JavaScript de WebKit y se enfoca en velocidades de ejecución y tiempos de inicio rápidos.



# Lecturas recomendadas

---



Deno v. Oracle (arco en desarrollo sobre liberar la marca JavaScript):

- [javascript.tm](https://javascript.tm): “Oracle, it’s time to free JavaScript”. Carta abierta a Oracle.
- [Deno v. Oracle: Canceling the JavaScript Trademark](#): Petición formal para cancelar la marca ante la USPTO
- [Oracle justified its JavaScript trademark with Node.js—now it wants that ignored](#): Oracle pide a la USPTO descartar la petición

# Agenda

---

Node.js

✨ **Arquitectura de Node**

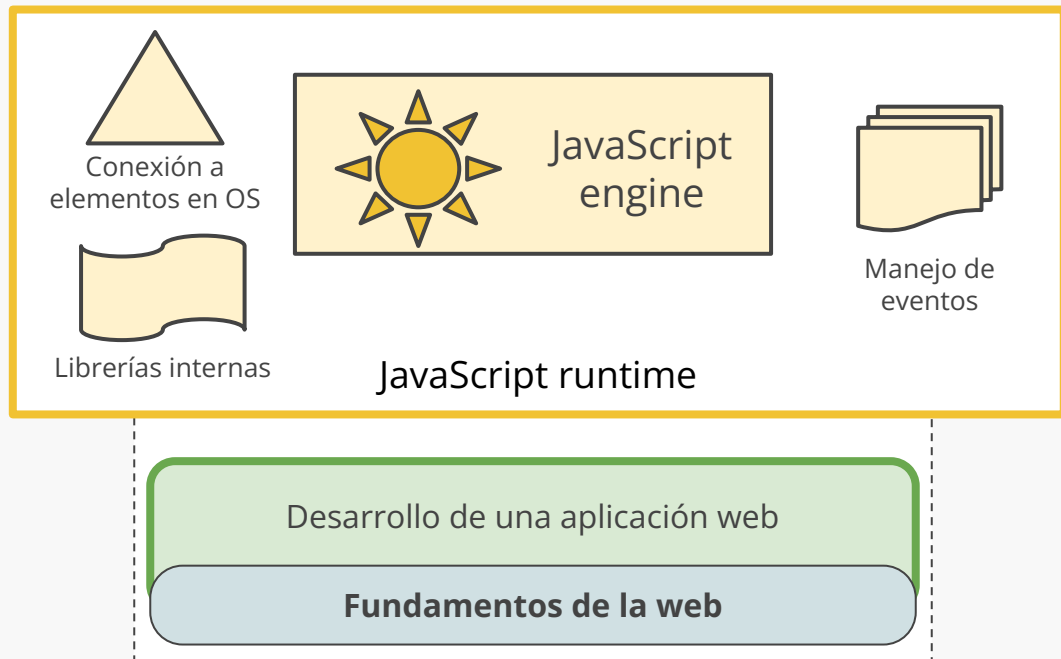
Funcionamiento interno

Usando Node.js



# Contenidos del curso

---



# Arquitectura de Node.js

---

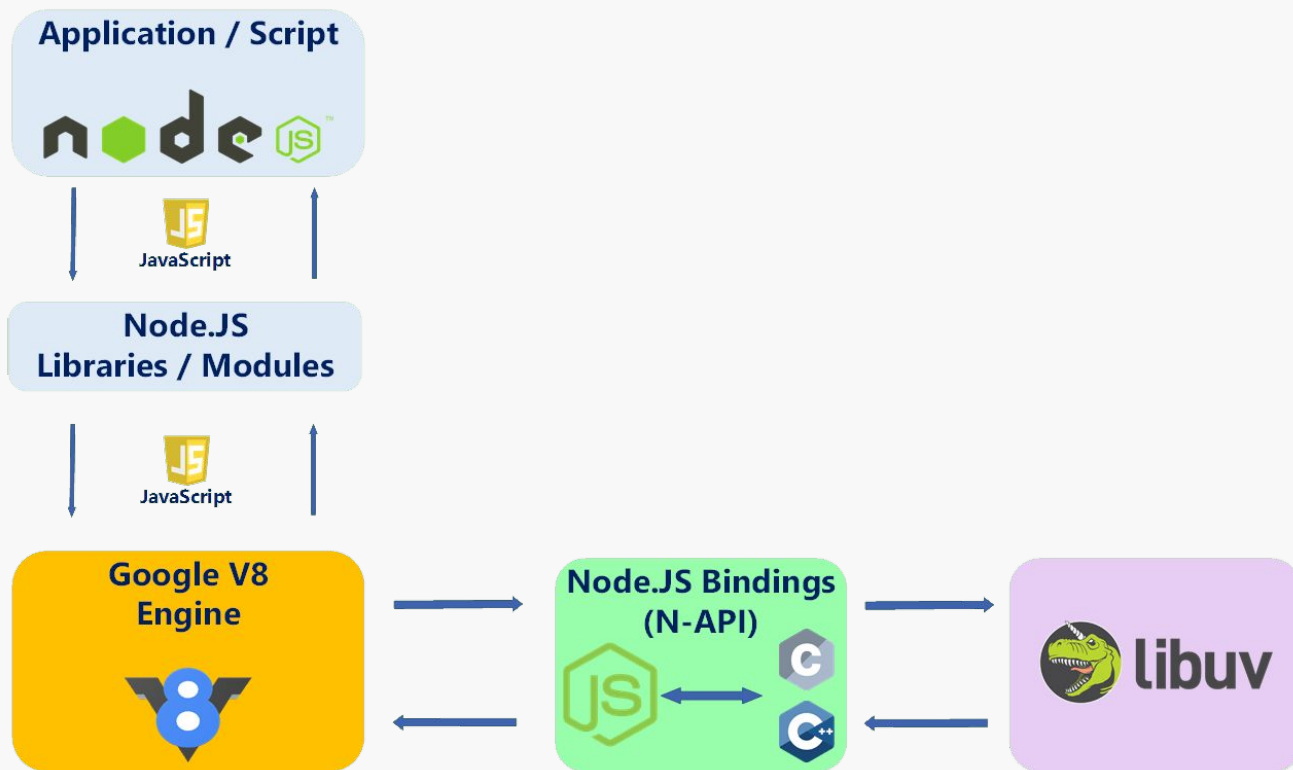
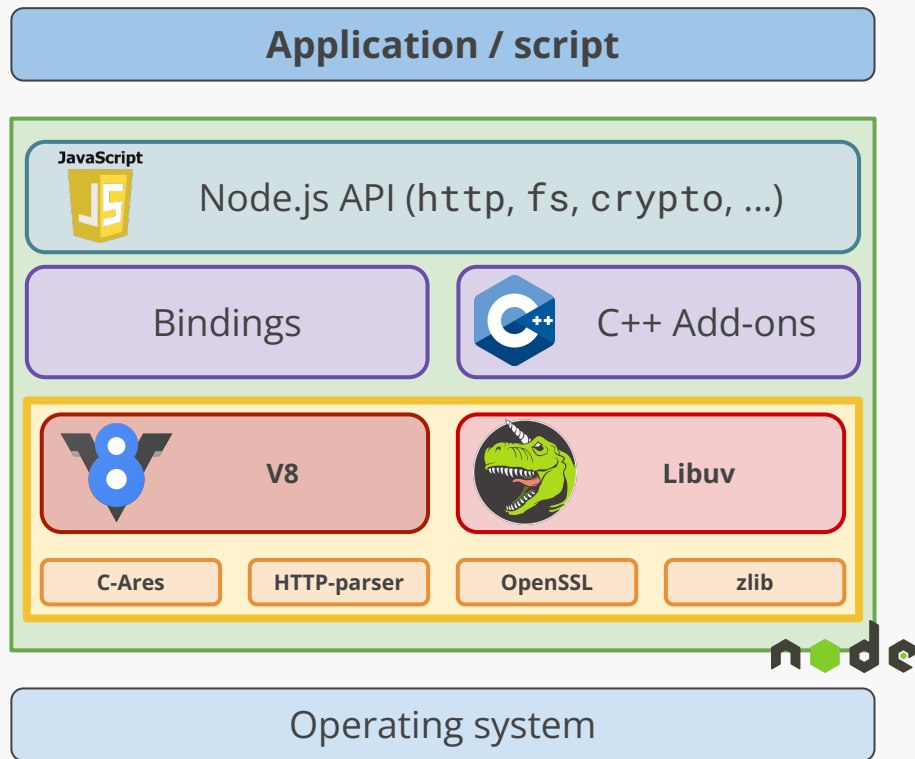


Figura: [Node.js Inside Out A look at Node.js architecture and its impact on code performance | IMELGRAT.ME](#) (Dead link)

# Arquitectura de Node.js

La arquitectura de NodeJS está diseñada como un conjunto de capas de abstracción, donde el código de quien programa se encuentra arriba y cada capa utiliza la API disponible por la capa de abajo

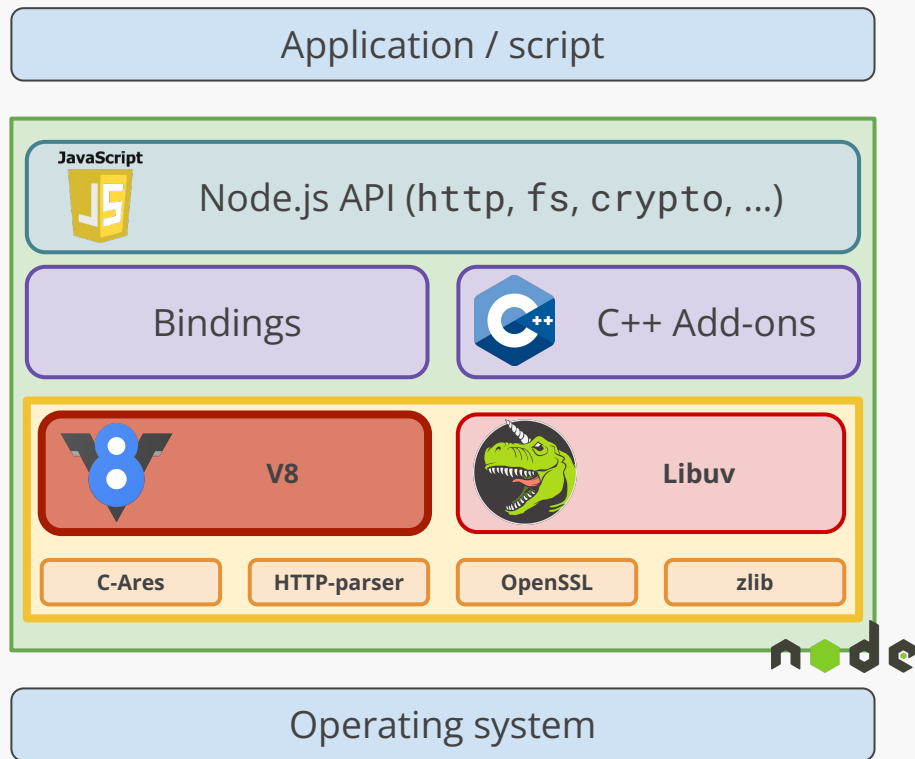


# Arquitectura de Node.js

## Motor V8

Es el *runtime environment* en el que se ejecuta el código JavaScript

Es un compilador JIT (Just-in-time), escrito en C++, que salta entre compilar y ejecutar bloques de código continuamente

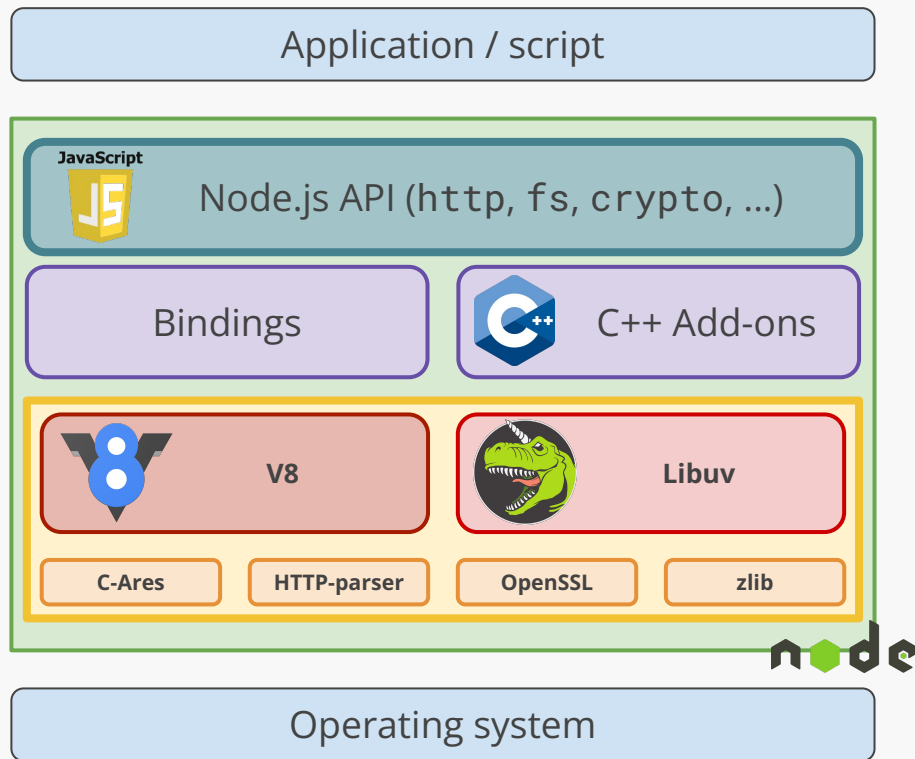


# Arquitectura de Node.js

## Node.js API

Lista de métodos ofrecidos por Node que pueden ser usados por quien programa (http, crypto, fs, net, .etc)

Lista completa disponible en la [documentación](#). API escrita en JavaScript.

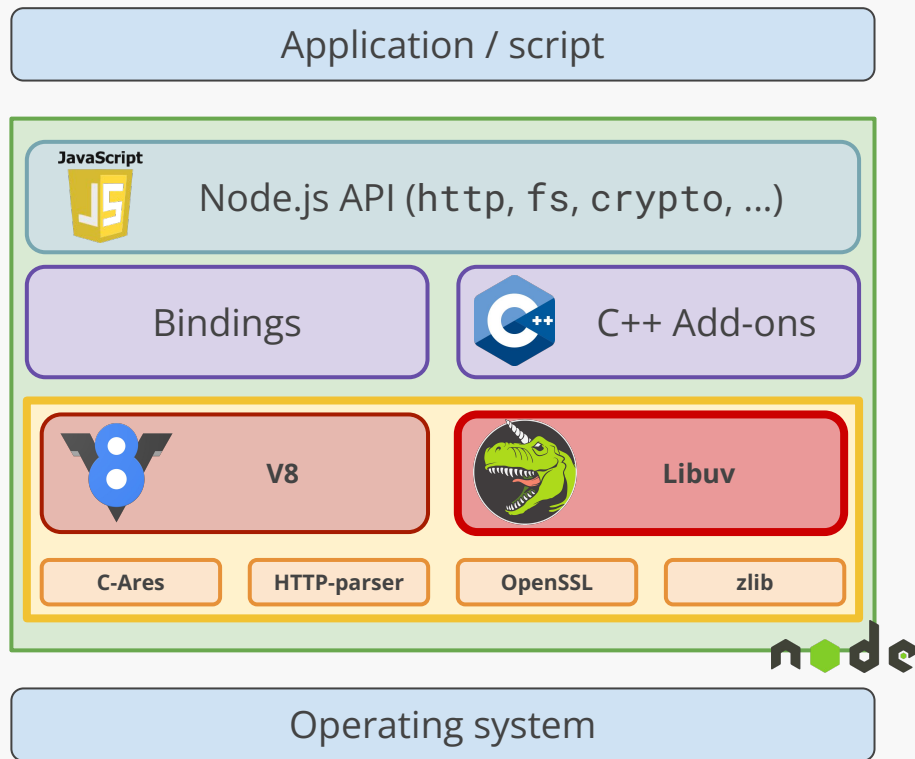


# Arquitectura de Node.js

## libuv

Librería multi-plataforma inicialmente desarrollada para Node.js, desarrollada en C.

Se especializa en abstraer operaciones I/O para que sean non-blocking en múltiples plataformas

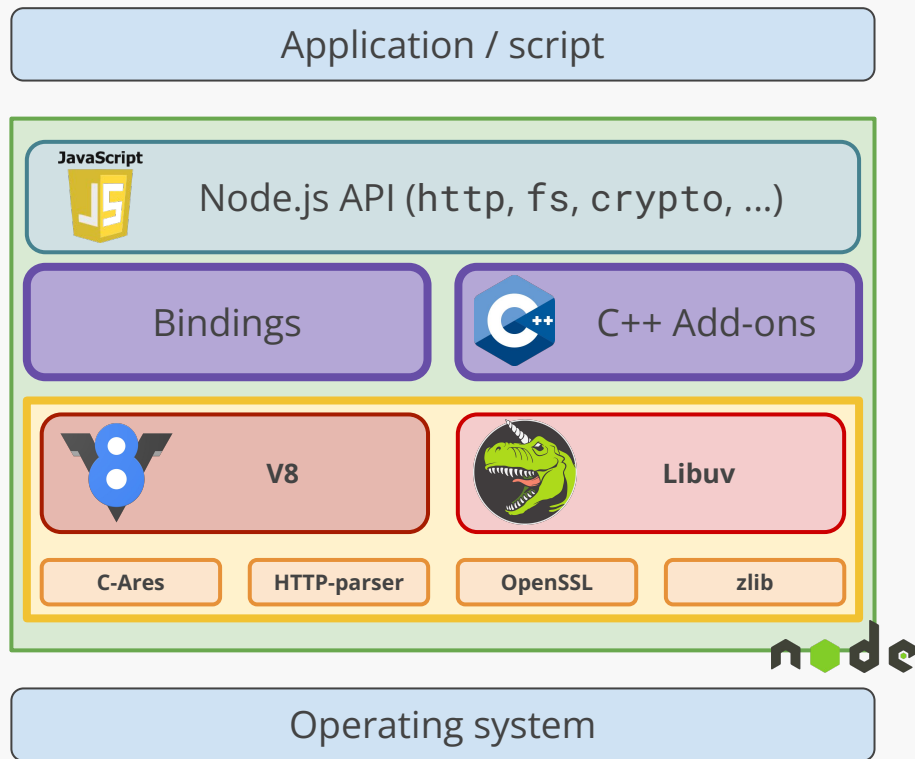


# Arquitectura de Node.js

## Bindings y C++ add-ons

Dado que V8 está escrito en C++, libuv en C y así, es necesario implementar *bindings* para permitir que el código en JS de la API utilice esta capa.

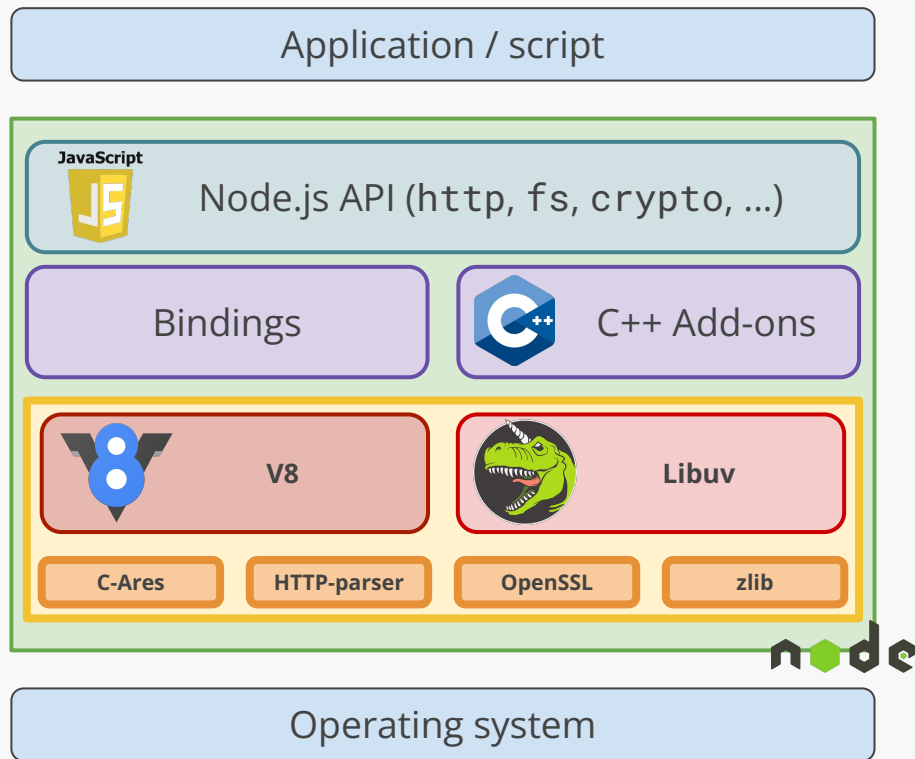
Es más, podemos implementar nuestros propios módulos (add-ons) en C++ para ser usados en Node ([docs](#))



# Arquitectura de Node.js

## Dependencias de Node

- **C-Ares:** *requests* DNS asíncronas
- **HTTP-parser:** parsing de respuestas y solicitudes HTTP
- **OpenSSL:** funciones criptográficas básicas como SSL, TLS y otras
- **zlib:** para rápida compresión y descompresión





# En resumen...

---

## NodeJS

- *Runtime* para JavaScript escrito en C++
- Puede interpretar y ejecutar JavaScript
- Incluye soporte para la API de NodeJS



## Node-API (API de NodeJS)

- Conjunto de librerías en JavaScript útiles para crear programas

## V8 (de Google)

- Intérprete de JavaScript (motor) que NodeJS usa para interpretar, compilar y ejecutar código de JavaScript



# Agenda

---

Node.js

Arquitectura de Node



**Funcionamiento interno**

Usando Node.js

## (CPU y I/O, *blocking* y *non-blocking*.)

---

- I/O se refiere a operaciones de transferencia de datos desde o hacia el computador (lectura de archivos, *networking*)
- CPU se refiere a operaciones de cálculo de valores/datos

**Node.js es lento en operaciones CPU (*single thread*) pero muy bueno en operaciones I/O (soporte directo, sin extras)**

- *Blocking* son las operaciones que bloquean la ejecución de otras operaciones hasta que esta termine
- *Non-blocking* son las operaciones que no detienen la ejecución de código

**En Node.js, esto se refiere a ejecución de código JavaScript y código que no depende de JavaScript (por ejemplo... operaciones I/O)**

# Single thread

---

JavaScript es *single-threaded*:

- Ejecuta una pieza de código a la vez
- El código a ejecutar se pone en la cola de eventos

En el ejemplo, cuando se pincha el botón la función asignada a `onclick` se agrega a la cola

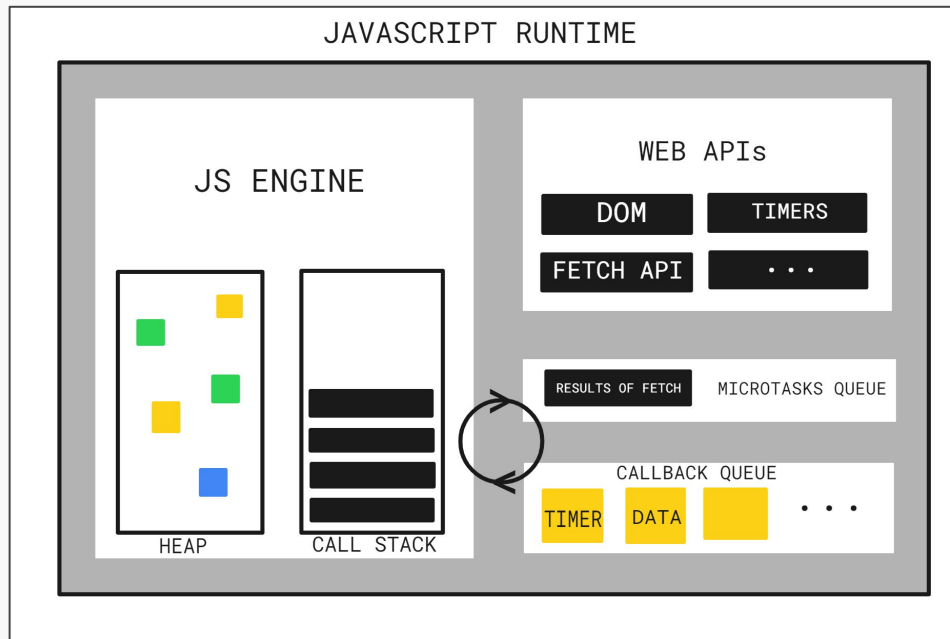
```
let button = document.getElementById('my-btn');  
button.onclick = function (event) {  
    console.log('Clicked');  
};
```

# Single thread

JavaScript es *single-threaded*:

- Ejecuta una pieza de código a la vez
- El código a ejecutar se pone en la cola de eventos

En el ejemplo, cuando se pincha el botón la función asignada a `onclick` se agrega a la cola

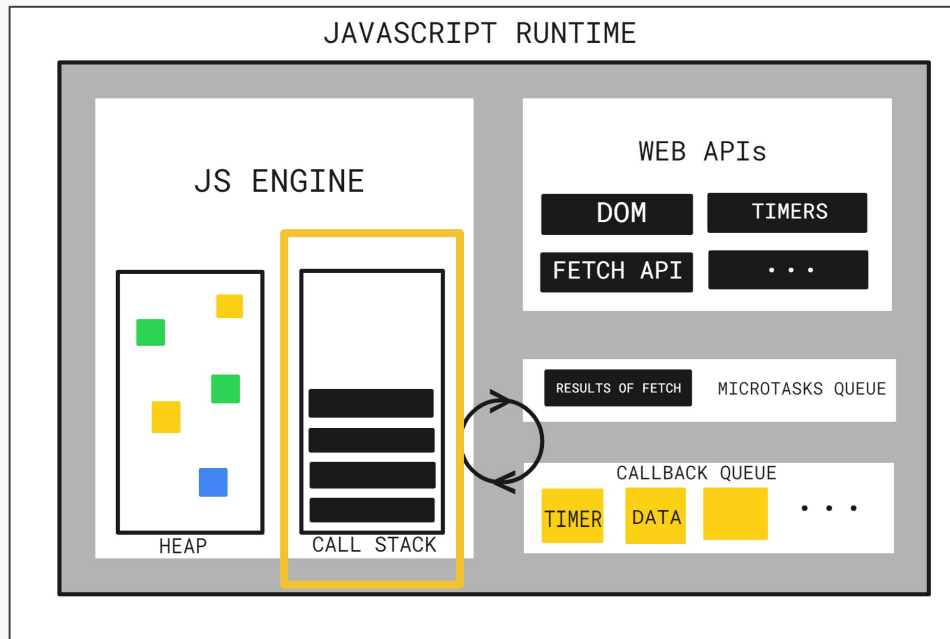


# Single thread

JavaScript es *single-threaded*:

- Ejecuta una pieza de código a la vez
- El código a ejecutar se pone en la cola de eventos

En el ejemplo, cuando se pincha el botón la función asignada a `onclick` se agrega a la cola



# ***Blocking code*** - Ejecución de código en orden

---

Recordemos que Javascript es *single-threaded*, o sea que irá línea por línea ejecutando el código descrito a continuación:

```
function funcionEjemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
funcionEjemplo();
```

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Al ejecutar el archivo, se ejecuta la función `main()`, representando la ejecución de todo el archivo. Agregamos `main()` al *call stack*

Call stack

main()



# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo(); ←
```

El primer llamado en ejecutarse es a la función `ejemplo()`.  
Agregamos la función en lo alto del *stack*, pues actúa de forma LIFO

Call stack

ejemplo()

main()

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Luego se llama a la función `console.log()` para imprimir un mensaje. Repetimos el procedimiento

Call stack

`console.log()`

`ejemplo()`

`main()`

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Una vez completada la función que hemos agregado al *call stack*, procedemos a quitarla de la estructura

Call stack

console.log()

ejemplo()

main()

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Una vez completada la función que hemos agregado al *call stack*, procedemos a quitarla de la estructura

*Call stack*

ejemplo()

main()

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Nos encontramos con una nueva llamada a `console.log`, por lo que agregamos esta función al call stack

Call stack

console.log()

ejemplo()

main()

# Call stack: siguiendo la ejecución

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Una vez completada la función, la quitamos del *stack*

Call stack

console.log()

ejemplo()

main()

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Una vez completada la función, la quitamos del *stack*

Call stack

ejemplo()

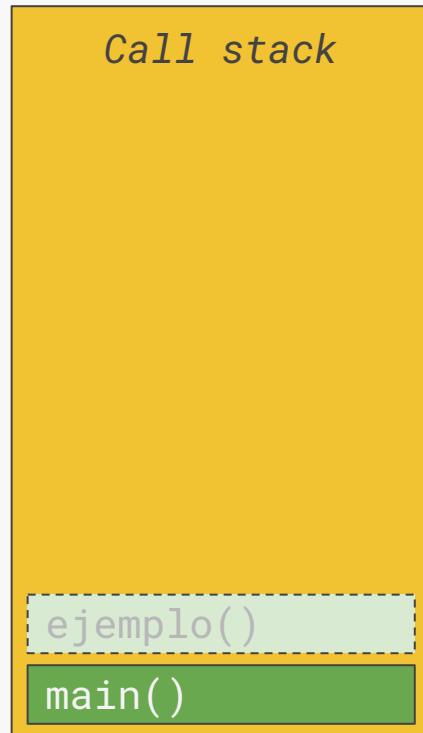
main()

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo(); ←
```

Ya no queda más código por ejecutar de la función `ejemplo`, por lo que podemos quitar esta función del *call stack*





# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Ya no queda más código por ejecutar de la función `ejemplo`, por lo que podemos quitar esta función del *call stack*

*Call stack*

`main()`

# Call stack: siguiendo la ejecución

---

```
function ejemplo() {  
  console.log('Hello');  
  for (let i = 0; i < 2 * 1000000000; i += 1) {  
    // ...  
  }  
  console.log('Bye');  
}  
  
ejemplo();
```

Con esto, acaba la ejecución de todo el archivo

*Call stack*

# *Non-blocking code* - Código asíncrono

---

¿Y si queremos ejecutar una operación que no sabemos cuánto va a tardar?

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

# Ya hemos usado *callbacks*...

---

```
element.addEventListener('click', event => {  
  // ...  
});
```

La función entregada como *event listener* va a ser llamada cuando ocurra el evento, por eso el nombre... va a ser “called back”

# Callbacks

Una función de *callback* es una función que se pasa a otra función como un argumento, que luego se invoca dentro de la función externa para completar algún tipo de rutina o acción

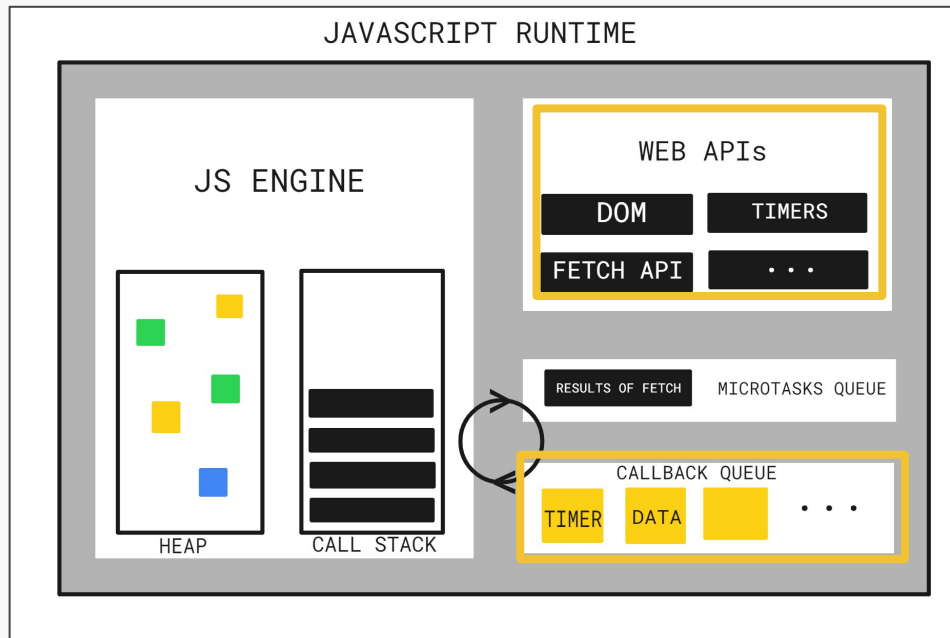
```
function primeraFuncion() {  
    // Hacer algo  
}  
  
function otraFuncion(callback) {  
    // Hacer algo  
    callback(); // Llamada al callback  
}  
  
otraFuncion(primerFuncion);
```

# ¿Dónde queda la función?

JavaScript es *single-threaded*:

- Ejecuta una pieza de código a la vez
- El código a ejecutar se pone en la cola de eventos

En el ejemplo, cuando se pincha el botón la función asignada a `onclick` se agrega a la cola



# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Al igual que antes, partimos con la función que representa la ejecución del archivo actual: `main()`

*Call stack*

`main()`

# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Agregamos al *stack* la ejecución de la función `setTimeout`

*Call stack*

`setTimeout()`

`main()`

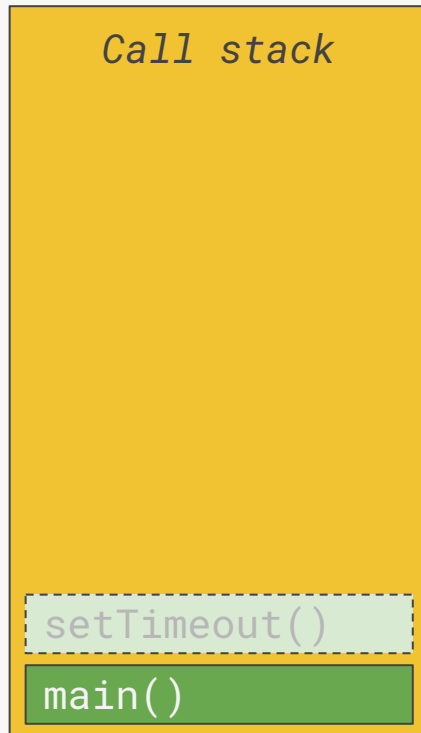


# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez que se completa su ejecución (“instantánea”), podemos quitar a `setTimeout` del *call stack*



# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez que se completa su ejecución (“instantánea”), podemos quitar a `setTimeout` del *call stack*

*Call stack*

`main()`

# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

... ahora esperamos, mientras no se ejecuta ninguna línea de código

Call stack



# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

... y una vez completados los 3 segundos se llama a nuestra función *callback* anónima

Call stack

callback()

# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Se produce el llamado a `console.log`

*Call stack*

`console.log()`

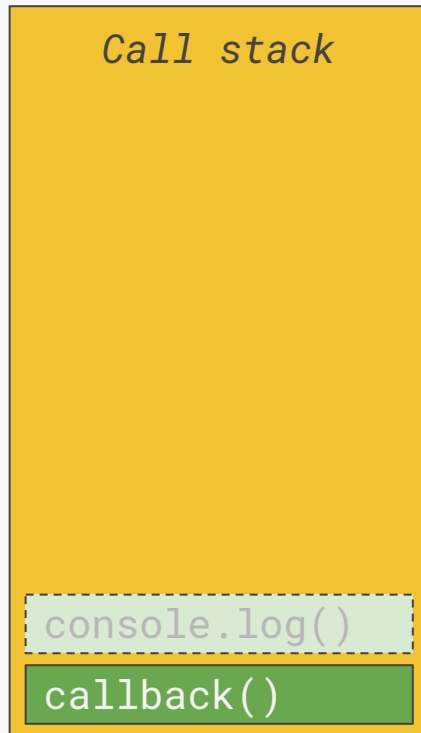
`callback()`

# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez completada la función, la quitamos del *call stack*



# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez completada la función, la quitamos del *call stack*

*Call stack*

callback()

# Call stack: siguiendo la ejecución

---

```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez completada la ejecución de todo nuestro *callback*, lo quitamos también del *call stack*

Call stack

callback()



# Call stack: siguiendo la ejecución

---

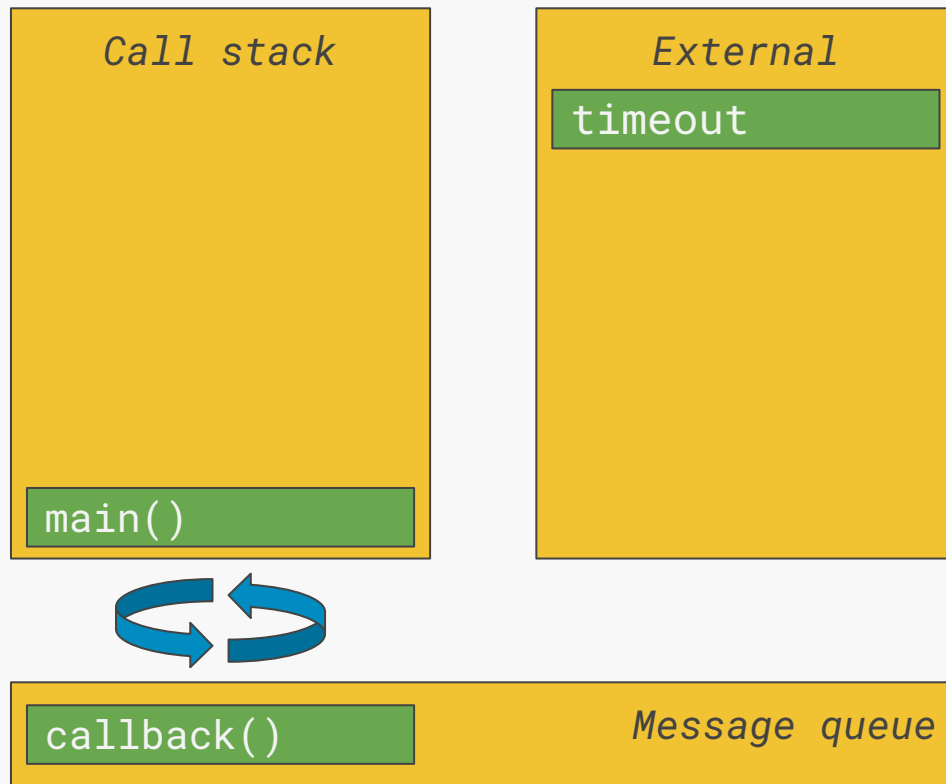
```
setTimeout(() => {  
  console.log('Pasaron 3 segundos');  
}, 3000);
```

Una vez completada la ejecución de todo nuestro *callback*, lo quitamos también del *call stack*

*Call stack*

# Event loop

1. Llamadas asíncronas son **manejadas por componentes externos**
2. Componentes externos **agregan llamadas a una cola**
3. Thread principal **continúa ejecutando código**
4. **Cuando se vacía el *call stack***, se toman elementos de la cola y se ejecutan
5. Ejecución es en el **orden en que fueron encolados**



# Modelo de concurrencia y loop de eventos - JavaScript | MDN:

## Lectura recomendada sobre el funcionamiento del *loop* de eventos en JavaScript

Esta página ha sido traducida del inglés por la comunidad. Aprende más y únete a la comunidad de MDN Web Docs.

Filter

### References

- Built-in objects
- Expressions & operators
- Statements & declarations
- Functions
- Classes
- Regular expressions
- Errors
- ▼ Misc

JavaScript technologies overview

Execution model

Lexical grammar

Iteration protocols

Strict mode

Template literals

## Modelo de concurrencia y loop de eventos

JavaScript posee un modelo de concurrencia basado en un "loop de eventos". Este modelo es bastante diferente al modelo de otros lenguajes como C o Java.

### Conceptos de un programa en ejecución

Las siguientes secciones explican un modelo teórico. Los motores modernos de JavaScript implementan y optimizan fuertemente la semántica descrita a continuación.

### Representación visual

Stack, heap, queue

### Pila (Stack)

Las llamadas a función forman una pila de *frames*. Un frame encapsula información como el contexto y las variables locales de una función.

### En este artículo

- Conceptos de un programa en ejecución
- Loop de eventos
- Nunca se interrumpe

# Heap y call stack

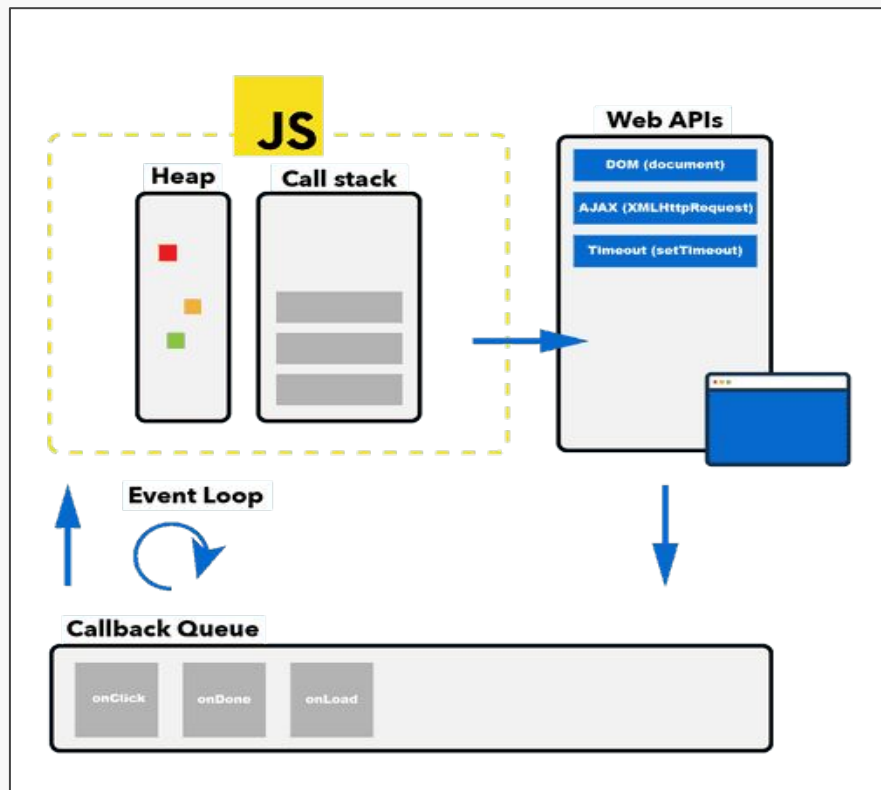


Figura: [JavaScript Event Loop And Call Stack Explained](#) | Feliz Gerschau

## JavaScript Event Loop And Call Stack Explained | Felix Gerschau:

Lectura recomendada sobre cómo opera JavaScript en el navegador como entorno de ejecución

**Felix Gerschau**

[Articles](#)

[About](#)

JAVASCRIPT

# JavaScript Event Loop And Call Stack Explained

Learn how JavaScript works in the browser: In this article, I explain how the call stack, event loop, job queue and more work together.

# ¿Cómo funciona Node.js durante la ejecución?

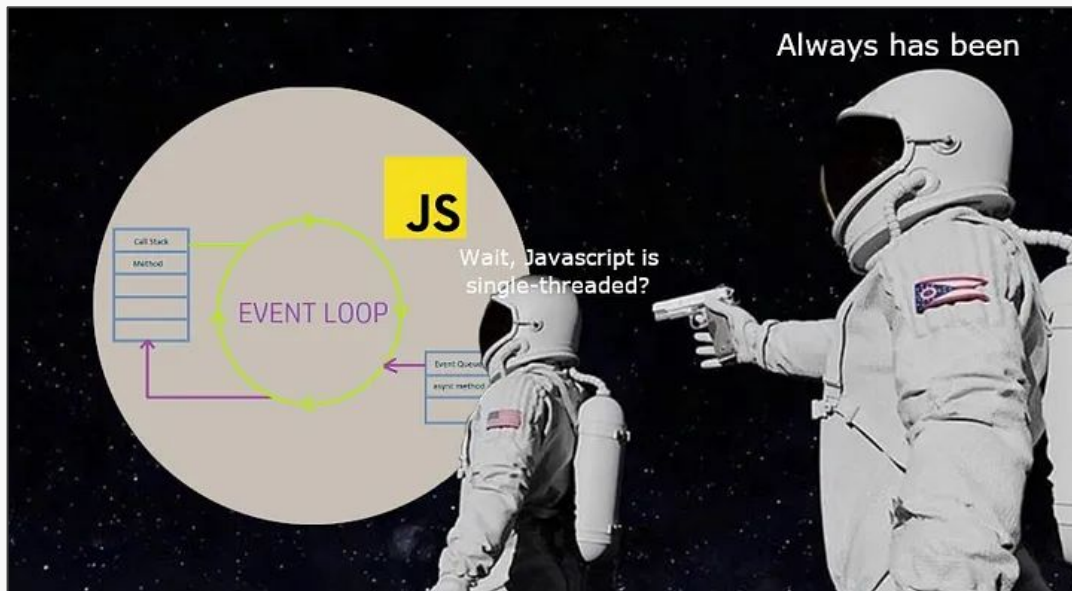
---

Las aplicaciones en Node.js ocupan una arquitectura **“Single-Threaded with Event Loop Model”**. Su funcionamiento está inspirado en el funcionamiento basado en eventos de Javascript que podemos encontrar en el navegador. Esto permite realizar operaciones *non-blocking* en I/O

En simple, esto significa que **solo una línea de código está en ejecución** en cualquier momento. Al mismo tiempo, existen *threads* adicionales que ejecutan otras instrucciones de manera asíncrona, esperando el resultado de las operaciones que sí son *blocking* en I/O

# ¿Cómo funciona Node.js durante la ejecución?

Recordar que tenemos dos componentes en juego: **V8** que ejecuta el código en JavaScript es síncrono, y **libuv** que abre paso a la ejecución asíncrona. Por lo tanto, en esencia, **JavaScript es síncrono** (ejecuta una línea de código a la vez)



# ¿Cómo funciona Node.js durante la ejecución?

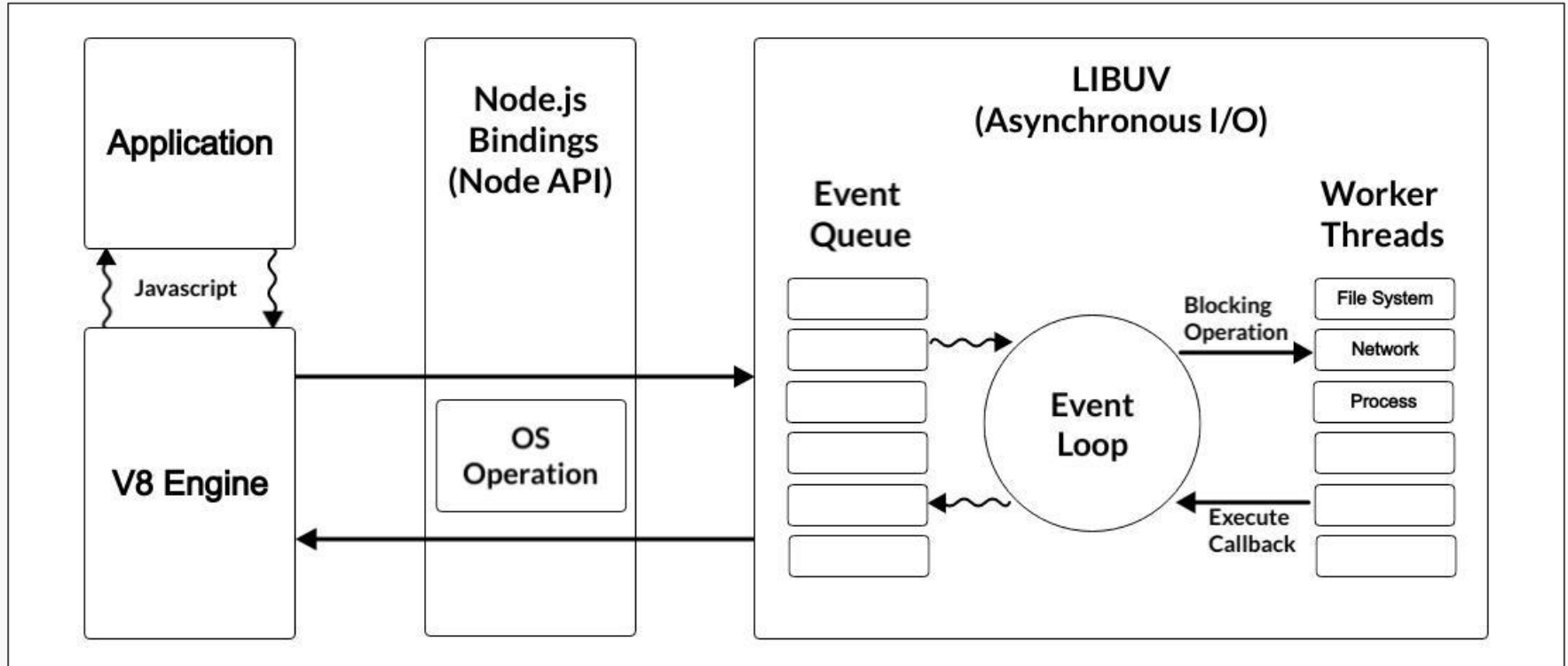
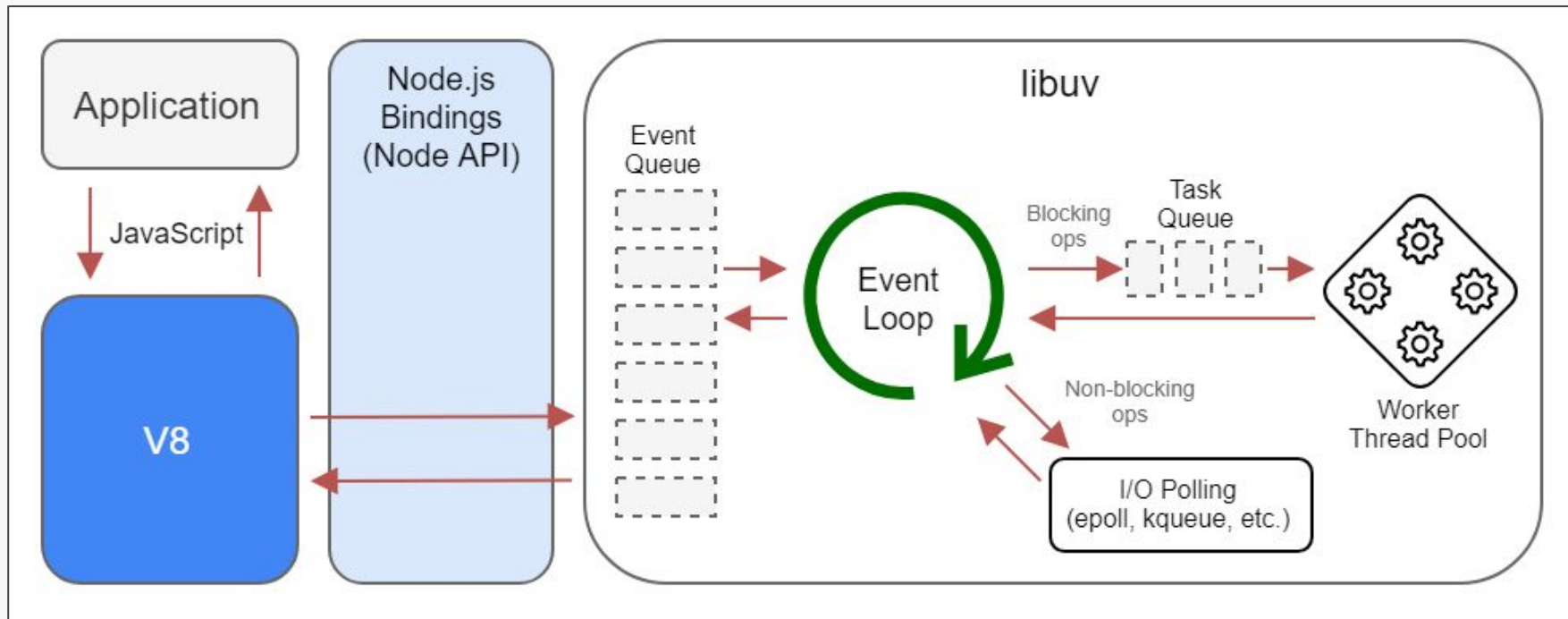


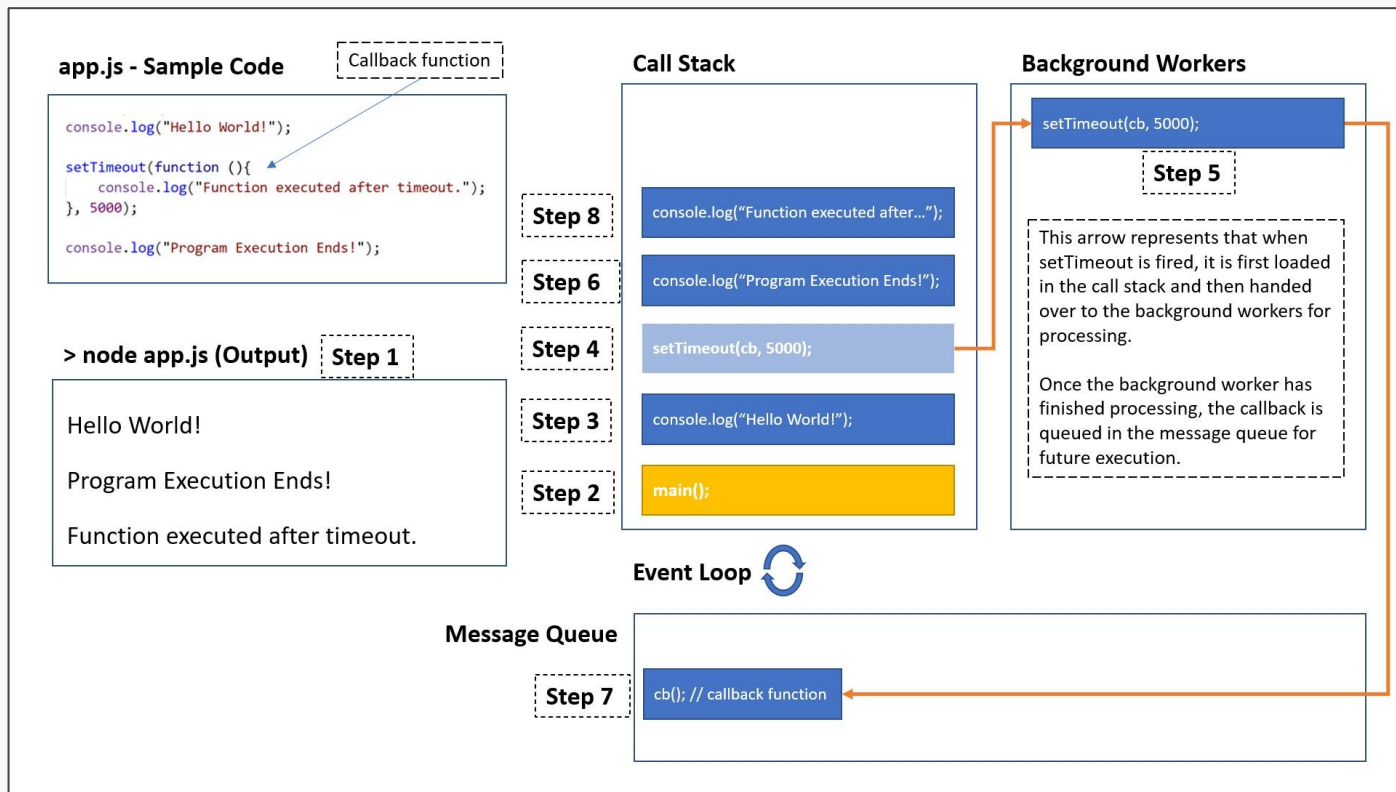
Figura: [Is Node.js entirely based on a single-thread ?](#) | [GeeksforGeeks](#)



# Event loop



# Event loop (Otra visualización)



- LIBUV

# Agenda

---

Node.js

Arquitectura de Node

Funcionamiento interno



**Usando Node.js**

# Comando node

---

El comando **node**, sin un nombre de archivo, **abre un REPL** (una consola del lenguaje) similar a la consola del navegador o cuando ejecutamos los comandos `irb` o `python`:

```
> node
Welcome to Node.js v23.10.0.
Type ".help" for more information.
>
>
> let x = 5;
undefined
> x++
5
> x
6
```

# Ejecución de *scripts*

---

NodeJS puede ser usado para escribir y ejecutar *scripts* en JavaScript

```
function imprimirPoema() {  
  const verso = `  
    Este código por IA ha sido creado,  
    Con precisión y arte bien elaborado.  
    Las palabras en pantalla ahora ves,  
    Un poema que una máquina compuso esta vez.  
  `;  
  console.log(verso);  
}  
  
imprimirPoema();  
imprimirPoema();
```

# Ejecución de *scripts*

---

El comando node también puede ser utilizado para ejecutar *scripts*

```
> node ejemplo.js
```

```
Este código por IA ha sido creado,  
Con precisión y arte bien elaborado.  
Las palabras en pantalla ahora ves,  
Un poema que una máquina compuso esta vez.
```

```
Este código por IA ha sido creado,  
Con precisión y arte bien elaborado.  
Las palabras en pantalla ahora ves,  
Un poema que una máquina compuso esta vez.
```

# NodeJS del lado del servidor

---

La propuesta de Node.js es poder usar JS desde el servidor, por ejemplo:

```
const http = require('http');

const server = http.createServer();

server.on('request', (req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.on('listening', () => {
  console.log('Server running!');
});

server.listen(3000);
```



# require()

---

La declaración `require()` en NodeJS carga un módulo, similar al uso que le damos a `import` en Python. Podemos hacer `require()` de módulos incluidos con NodeJS o módulos que escribamos nosotros

En este ejemplo, 'http' es el [módulo HTTP de NodeJS](#), y el objeto retornado es usado para hacer llamadas a su API ([http.createServer\(\)](#))

```
const http = require('http');
```

```
const server = http.createServer();
```

```
server.on('request', (req, res) => {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello World\n');
```

# NodeJS del lado del servidor

---

```
const http = require('http');  
  
const server = http.createServer();
```

```
server.on('request', (req, res) => {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello World\n');  
});
```

```
server.on('listening', () => {  
  console.log('Server running!');  
});  
  
server.listen(3000);
```

La función [on\(\)](#) es el equivalente de NodeJS a `addEventListener`

El evento [request](#) se emite cada vez que existe una nueva solicitud HTTP para que el programa en NodeJS procese.

- **req**: contiene información de la *request* entrante
- **res**: respuesta sobre la que podemos escribir

# NodeJS del lado del servidor

---

```
const http = require('http');

const server = http.createServer();

server.on('request', (req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});
```

```
server.on('listening', () => {
  console.log('Server running!');
});

server.listen(3000);
```

La función [listen\(\)](#) empezará a aceptar conexiones en el número de puerto dado (3000)

El evento [listening](#) se emite cuando el servidor sea “atado” (*bounded*) a un puerto

... ¿puerto? ¿*binding*?

# Puertos y *binding*

---

**Puerto/port:** En el contexto de *networking*, es un lugar donde se da una conexión lógica, y se describe con un número entre 0 y 65535 (número de 16 bits)

Cuando el proceso de un servidor se inicia, se le indica al sistema operativo **cuál número de puerto asociar con dicho servidor**. Esto es *binding*

Las conexiones (como TCP) deben indicar a qué puerto e IP se quieren conectar. Existen muchos puertos que son usados por defecto para para protocolos particulares

|                                                                   |
|-------------------------------------------------------------------|
| 21: File Transfer Protocol (FTP)                                  |
| 22: Secure Shell (SSH)                                            |
| 23: Telnet remote login service                                   |
| 25: Simple Mail Transfer Protocol (SMTP)                          |
| 53: Domain Name System (DNS) service                              |
| 80: Hypertext Transfer Protocol (HTTP) used in the World Wide Web |
| 110: Post Office Protocol (POP3)                                  |
| 119: Network News Transfer Protocol (NNTP)                        |
| 123: Network Time Protocol (NTP)                                  |
| 143: Internet Message Access Protocol (IMAP)                      |
| 161: Simple Network Management Protocol (SNMP)                    |
| 194: Internet Relay Chat (IRC)                                    |
| 443: HTTP Secure (HTTPS)                                          |

# ¿Cuáles limitaciones tiene nuestro servidor?

---

```
const http = require('http');

const server = http.createServer();

server.on('request', (req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.on('listening', () => {
  console.log('Server running!');
});

server.listen(3000);
```

## Express - Node.js web application framework:

Página principal de Express, un framework para aplicaciones web en JavaScript bastante popular

express

 search

**Home**

Getting started

Guide

API reference

Advanced topics

Resources

Support

Blog



English



# Express

[5.1.0](#)

Fast, unopinionated,  
minimalist web  
framework for [Node.js](#)

```
$ npm install express --save
```

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```



**Express@5.1.0: Now the Default on npm with LTS Timeline**

Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. [Check out our latest blog for more information.](#)

## Express - Node.js web application framework:

Página principal de Express, un framework para aplicaciones web en JavaScript bastante popular

express

 search

Home

Getting started

Guide

API reference

Advanced topics

Resources

Support

Blog



English



Express  
Fast, unopinionated,  
minimalist web  
framework

\$ npm install express -

```
const express = require('express');  
const app = express();  
const port = 3000;
```

```
app.get('/', (req, res) => {  
  res.send('Hello World!');  
});
```

```
app.listen(port, () => {  
  console.log(`Example app listening on port ${port}`);  
});
```



**Express@5.1.0: Now the Default on npm with LTS Timeline**

Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. [Check out our latest blog for more information.](#)

## Koa - next generation web framework for node.js:

Página principal de Koa, un framework web minimalista para Node.js



# koa

next generation web framework for node.js



## Koa - next generation web framework for node.js:

Página principal de Koa, un framework web minimalista para Node.js



```
const Koa = require('koa');
const app = new Koa();

app.use(async ctx => {
  ctx.body = 'Hello World';
});

app.listen(3000);
```

# *Package managers: módulos en JavaScript*

---

Así como Python tiene pip y Ruby tiene gemas...



# npm: Node Package Manager

---

CLI que permite instalar y manejar paquetes, escrita en JS y compatible con NodeJS. Los paquetes de este índice son tomados desde el repositorio disponible en [el sitio de NPM](https://www.npmjs.com/)

```
npm install package-name
```

Descarga el paquete en una carpeta `node_modules`  
y queda disponible para ser usado en archivos JS

```
npm uninstall package-name
```

Remueve la librería de la carpeta `node_modules`  
y borra la carpeta si es necesario



# Yarn

---

Al igual que npm, [yarn](#) es un *package manager* (pero *open source*) que ofrece una CLI para manejar paquetes, pero incluye mejoras como mayor *performance*, resolución de dependencias determinística, instalación de paquetes en paralelo y más, usando el mismo índice de paquetes que utiliza npm

```
yarn add package-name
```

Agrega una dependencia al proyecto actual

```
yarn remove package-name
```

Remueve la dependencia del proyecto



# *Spoilers:* ¿Cómo se verá nuestro servidor en Koa?

---

```
const Koa = require('koa');
const Router = require('@koa/router');

const app = new Koa();
const router = new Router();

router.get('/hello', (ctx, next) => {
  ctx.body = 'world!';
});

app
  .use(router.routes())
  .use(router.allowedMethods());

app.listen(3000);
```

# Agenda

---

Node.js

Arquitectura de Node

Funcionamiento interno

Usando Node.js



**¿Qué se viene pronto?**

# Próximas fechas importantes: clases y ayudantías

|                                         | Clases y ayudantías |                                         |           |                               |                                 |              |                         |
|-----------------------------------------|---------------------|-----------------------------------------|-----------|-------------------------------|---------------------------------|--------------|-------------------------|
|                                         | Lunes               | Martes                                  | Miércoles | Jueves                        | Viernes                         | Sábado       | Domingo                 |
| <b>Semana 4</b><br><b>(24/3 - 30/3)</b> |                     | React                                   |           | Exploración y consumo de APIs | Ayudantía: React                |              |                         |
| <b>Semana 5</b><br><b>(31/3 - 06/4)</b> |                     | HOY                                     |           | Desarrollando servidores      | Ayudantía: Consumo de APIs      |              |                         |
| <b>Semana 6</b><br><b>(07/4 - 13/4)</b> |                     | Construcción de APIs                    |           | QA, CI & CD                   | Ayudantía: Construcción de APIs |              |                         |
| <b>Semana 7</b><br><b>(14/4 - 20/4)</b> |                     | Fundamentos de front-end                |           | Jueves Santo                  | Viernes Santo                   | Sábado Santo | Domingo de Resurrección |
| <b>Semana 8</b><br><b>(21/4 - 27/4)</b> |                     | Charla Front-end (Florencia Valladares) |           | Diseño web en la práctica     |                                 |              |                         |

# Próximas fechas importantes: evaluaciones

|                                  | Evaluaciones  |               |              |              |                 |              |                         |
|----------------------------------|---------------|---------------|--------------|--------------|-----------------|--------------|-------------------------|
|                                  | Lunes         | Martes        | Miércoles    | Jueves       | Viernes         | Sábado       | Domingo                 |
| <b>Semana 4</b><br>(24/3 - 30/3) | Control 1     |               |              |              | Inicio T1 React |              |                         |
| <b>Semana 5</b><br>(31/3 - 06/4) |               | HOY           |              |              | Fin T1 (React)  |              |                         |
| <b>Semana 6</b><br>(07/4 - 13/4) | Control 2     |               |              |              | Inicio T2 API   |              |                         |
| <b>Semana 7</b><br>(14/4 - 20/4) |               |               |              | Jueves Santo | Viernes Santo   | Sábado Santo | Domingo de Resurrección |
| <b>Semana 8</b><br>(21/4 - 27/4) | UC: Sin eval. | UC: Sin eval. | Fin T2 (API) |              |                 |              |                         |



Clase 7

# Node.js

---

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra  
aaossa@ing.puc.cl